Livrable 2

Lafon Lilian AIT Ali ilyass Vives Emma



Table des matières

Livrable 2 1
Rappel : 2
Peuplement de la base : 2
Réalisation des requêtes : 4
- Requête 1 : 4
- Requête 2 : 4
- Requête 3 : 5
- Requête 4 : 5
- Requête 5 : 6
- Requête 6 : 7
- Requête 7 : 7
- Requête 8 : 8
- Requête 9 : 9
- Requête 10 : 9
- Requête 11 : 10
- Requête 12 : 11

Exigence demander : 11
Conclusion : 12

Rappel :

A la suite de la création des différentes tables, nous avions comme demande la réalisation de 12 requêtes, pour que celle-ci puisse être opérationnel nous devions importer des données. Nous n'avions pas d'obligation pour la sélection des données, mais elles devaient pour certaines respecter une certaine quantité de données.

Peuplement de la base :

Afin de peupler la base nous avions fait appel à un générateur d'informations en essayant de respecter les requêtes pour qu'elle puisse renvoyer des résultats

Exemple : pour la requête 2 il faut prendre en compte les agences de bordeaux donc si dans nos données on n'importe pas d'agence à Bordeaux, cette requête ne renverra rien

Voici le code SQL utilisé pour l'importations de données

```
27 •  SET FOREIGN_KEY_CHECKS = 1;
28
29    -- ====================
30    -- REGIONS
31    -- ====================
32 •  INSERT INTO region (id_region, nom_region) VALUES
33    (1, 'Île-de-France'),
34    (2, 'Nouvelle-Aquitaine'),
35    (3, 'Occitanie'),
36    (4, 'Auvergne-Rhône-Alpes'),
37    (5, 'Bretagne');
38
```

```sql
-- ====================
-- AGENCES
-- ====================
INSERT INTO agence (id_agence, nom_agence, region_agence, nom_rue_agence, numero_rue_agence, code_postale_agence, id_ville) VALUES
(1, 'Agence Paris', 'IDF', 'Rue de Rivoli', 10, '75001', 1),
(2, 'Agence Bordeaux', 'NA', 'Rue Sainte-Catherine', 5, '33000', 2),
(3, 'Agence Toulouse', 'Occitanie', 'Rue d Alsace', 8, '31000', 3),
(4, 'Agence Lyon', 'ARA', 'Rue Mercière', 7, '69000', 4),
(5, 'Agence Rennes', 'Bretagne', 'Rue de la République', 6, '35000', 5),
(6, 'Agence Limoges', 'NA', 'Rue Jean Jaurès', 3, '87000', 6);
```

```sql
-- ====================
-- AGENTS
-- ====================
INSERT INTO agent (id_agent, nom_agent, prenom_agent, date_naissance_agent, date_embauche_agent, numero_rue_agent, nom_rue_agent, cod
(1, 'Martin', 'Jean', '1980-01-01', '2015-01-01', 1, 'Rue A', '75000', 1, 1),
(2, 'Dupont', 'Claire', '1990-02-02', '2020-03-01', 2, 'Rue B', '33000', 2, 2),
(3, 'Durand', 'Paul', '1985-03-03', '2018-04-01', 3, 'Rue C', '31000', 3, 3),
(4, 'Bernard', 'Sophie', '1992-04-04', '2021-05-01', 4, 'Rue D', '69000', 4, 4),
(5, 'Petit', 'Lucas', '1988-05-05', '2019-06-01', 5, 'Rue E', '35000', 5, 5),
(6, 'Robert', 'Emma', '1995-06-06', '2022-01-01', 6, 'Rue F', '75000', 1, 1),
(7, 'Richard', 'Hugo', '1987-07-07', '2017-02-01', 7, 'Rue G', '33000', 2, 2),
(8, 'Moreau', 'Julie', '1993-08-08', '2020-03-01', 8, 'Rue H', '31000', 3, 3),
(9, 'Simon', 'Louis', '1984-09-09', '2016-04-01', 9, 'Rue I', '69000', 4, 4),
(10, 'Laurent', 'Anna', '1996-10-10', '2023-01-01', 10, 'Rue J', '35000', 5, 5),
(11, 'Lefevre', 'Tom', '1989-01-01', '2020-01-01', 11, 'Rue K', '75000', 1, 1),
(12, 'Michel', 'Sarah', '1991-02-02', '2021-02-02', 12, 'Rue L', '33000', 2, 2),
(13, 'Garcia', 'Nina', '1994-03-03', '2022-03-03', 13, 'Rue M', '31000', 3, 3),
(14, 'David', 'Leo', '1986-04-04', '2019-04-04', 14, 'Rue N', '69000', 4, 4),
(15, 'Bertrand', 'Eva', '1997-05-05', '2023-05-05', 15, 'Rue O', '35000', 5, 5),
```

Voici quelque exemple d’importations nous avons veiller à respecter ces conditions d’importations : Vous devrez compléter ces données pour que votre base soit complète et que les requêtes soient pertinentes et génèrent un résultat (au moins 200 relevés de capteurs, au moins 20 employés, au moins 10 rapports...).

Vos données doivent cependant être réalistes quant aux régions et aux villes.

Pour les dates, vous pouvez vous baser sur un intervalle entre janvier 2017 et décembre 2024.

· En vous renseignant sur le sujet, vous comprenez que les quantités mesurées sont en ppm (partie par million). Mme Boyce ne semble pas exigeante sur les valeurs que vous mettrez dans la base de données. Mais vos recherches montrent que cela peut prendre des valeurs entre 0.01 et 500.

Comme nous pouvons le voir sur ces codes afin d’importer les données nous utilisons la fonction **insert into** puis nous précisions la table à fournir, puis **values** qui va annoncer les valeurs à importer en respectant bien le nombre d’entité d’une table il faut le même nombre de valeur

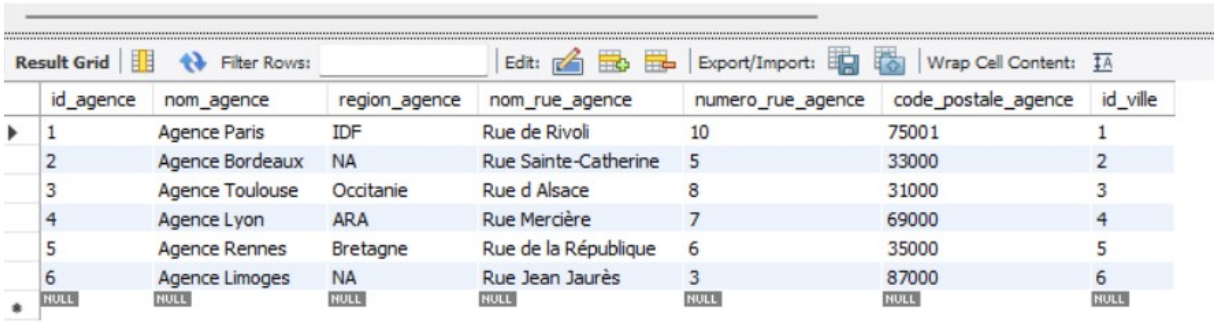
Si nous avons 4 entités dans une table il va falloir 4 valeurs dans la même ligne.

Nous avions cependant rencontré diverse difficulté notamment à l’exécution des requêtes car le peuplement de la base n’était pas parfait à chaque fois il a dû falloir modifier certaines fois le peuplement de la base pour que les requêtes s’exécutent sans difficultés.

Réalisation des requêtes :

Requête 1 :

```
1 •  SELECT *
2    FROM Agence;
```

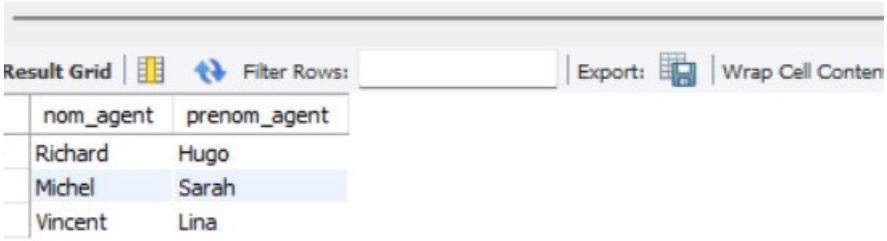


id_agence	nom_agence	region_agence	nom_rue_agence	numero_rue_agence	code_postale_agence	id_ville
1	Agence Paris	IDF	Rue de Rivoli	10	75001	1
2	Agence Bordeaux	NA	Rue Sainte-Catherine	5	33000	2
3	Agence Toulouse	Occitanie	Rue d Alsace	8	31000	3
4	Agence Lyon	ARA	Rue Mercière	7	69000	4
5	Agence Rennes	Bretagne	Rue de la République	6	35000	5
6	Agence Limoges	NA	Rue Jean Jaurès	3	87000	6
NULL	NULL	NULL	NULL	NULL	NULL	NULL

La requête récupère toutes les informations présentes dans la table agence. Elle permet d'obtenir une vue complète des agences enregistrées dans la base. Aucune jointure n'est nécessaire car toutes les données utiles sont contenues dans une seule table.

Requête 2 :

```
1 •  SELECT a.nom_agent, a.prenom_agent
2    FROM agent a
3    JOIN agent_technique t ON a.id_agent = t.id_agent
4    JOIN agence ag ON a.id_agence = ag.id_agence
5    WHERE ag.nom_agence = 'Agence Bordeaux';
6
```



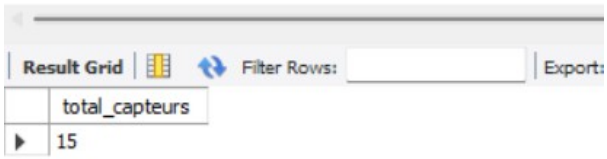
nom_agent	prenom_agent
Richard	Hugo
Michel	Sarah
Vincent	Lina

La requête permet de récupérer le nom et le prénom des agents qui sont à la fois rattachés à l'agence de Bordeaux et présents dans la table des agents techniques.

Elle utilise plusieurs tables pour croiser les informations : la table agent pour les identités, la table agent_technique pour filtrer uniquement les agents techniques, et la table agence pour identifier l'agence de rattachement. Le filtre sur le nom de l'agence permet de limiter le résultat aux agents travaillant à Bordeaux.

Requête 3 :

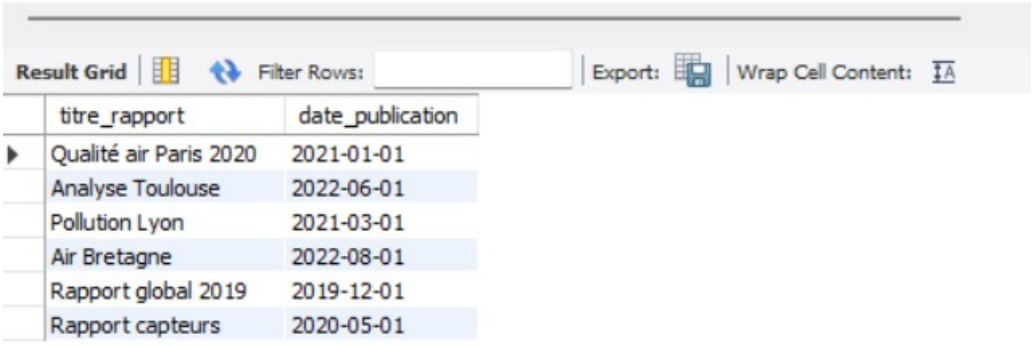
```
1 •  SELECT COUNT(*) AS total_capteurs
2    FROM capteur;
```



La requête compte le nombre total de capteurs présents dans la table **capteur**. Elle utilise COUNT(*) car cette fonction permet de compter toutes les lignes de la table sans exception. Aucune jointure ni condition n'est nécessaire, car l'objectif est simplement d'obtenir un total global des enregistrements.

Requête 4 :

```
1 •  SELECT titre_rapport, date_publication
2    FROM rapport
3    WHERE date_publication BETWEEN '2018-01-01' AND '2022-12-31';
```



La requête récupère le titre et la date de publication des rapports dont la date se situe entre le 1er janvier 2018 et le 31 décembre 2022. Elle utilise BETWEEN pour filtrer une plage de dates de manière simple et lisible, ce qui permet de

sélectionner uniquement les rapports publiés dans cette période sans avoir besoin de conditions multiples.

Requête 5 :

```sql
SELECT r.nom_region, m.date_mesure, m.valeur_ppm
FROM  region r
JOIN ville v ON v.id_region = r.id_region
JOIN agence ag ON ag.id_ville = v.id_ville
JOIN capteur c ON c.id_agence = ag.id_agence
JOIN gaz g ON c.id_gaz = g.id_gaz
JOIN mesure m ON m.id_capteur = c.id_capteur
WHERE g.sigle_gaz = 'CH4'
AND r.nom_region IN ('Île-de-France','Bretagne','Occitanie')
AND m.date_mesure IN ('2023-05-01','2023-06-01')
ORDER BY r.nom_region, m.date_mesure;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom_region	date_mesure	valeur_ppm
Bretagne	2023-05-01	140.80
Bretagne	2023-06-01	138.40
Île-de-France	2023-05-01	400.00
Île-de-France	2023-06-01	410.00
Occitanie	2023-05-01	155.50
Occitanie	2023-06-01	160.20

La requête récupère les valeurs de mesure du gaz CH_4 pour trois régions (Île-de-France, Bretagne et Occitanie) à deux dates précises (mai et juin 2023). Elle utilise plusieurs jointures pour relier les régions aux villes, aux agences, aux capteurs, puis aux mesures. Les conditions WHERE permettent de filtrer le gaz concerné, les régions et les dates souhaitées. Enfin, ORDER BY organise le résultat par région puis par date pour faciliter la lecture et la comparaison des valeurs.

Requête 6 :

```sql
SELECT DISTINCT a.nom_agent, a.prenom_agent
FROM agent a
JOIN agent_technique t ON a.id_agent = t.id_agent
JOIN responsable res ON a.id_agent = res.id_agent
JOIN capteur c ON res.id_capteur = c.id_capteur
JOIN gaz g ON c.id_gaz = g.id_gaz
WHERE g.type_gaz = 'GESI';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nom_agent	prenom_agent
Simon	Louis
Bertrand	Eva
Robert	Emma
Laurent	Anna

La requête permet d'obtenir la liste des agents (nom et prénom) qui sont à la fois agents techniques et responsables de capteurs mesurant des gaz de type GESI. Elle utilise plusieurs jointures pour relier les agents à leurs capteurs via la table responsable, puis aux gaz associés à ces capteurs. Le filtre WHERE limite les résultats aux gaz de type GESI. DISTINCT est utilisé pour éviter les doublons, car un même agent peut être responsable de plusieurs capteurs.

Requête 7 :

```sql
SELECT DISTINCT r.titre_rapport, r.date_publication
FROM rapport r
JOIN contient co ON r.id_rapport = co.id_rapport
JOIN mesure m ON co.id_mesure = m.id_mesure
JOIN capteur c ON m.id_capteur = c.id_capteur
JOIN gaz g ON c.id_gaz = g.id_gaz
WHERE g.sigle_gaz = 'NH3'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

titre_rapport	date_publication
Données NH3 et HFC	2024-05-15
NH3 analyse	2023-09-01

La requête permet d'obtenir les titres et dates de publication des rapports qui sont liés à des mesures du gaz NH_3 . Elle passe par plusieurs jointures pour relier les rapports aux mesures, puis aux capteurs et enfin aux gaz. Le filtre WHERE permet de ne conserver que les mesures concernant le NH_3. DISTINCT est utilisé pour éviter les doublons si un même rapport contient plusieurs mesures

concernées. Enfin, ORDER BY date_publication DESC classe les résultats du plus récent au plus ancien pour faciliter la lecture chronologique.

Requête 8 :

```sql
SELECT region, mois, concentration_moyenne_hfc
FROM (
    SELECT
        reg.nom_region AS region,
        DATE_FORMAT(m.date_mesure, '%Y-%m') AS mois,
        ROUND(AVG(m.valeur_ppm), 2) AS concentration_moyenne_hfc,
        RANK() OVER (
            PARTITION BY reg.id_region
            ORDER BY AVG(m.valeur_ppm) ASC
        ) AS rang_region
    FROM mesure m
    JOIN capteur c ON c.id_capteur = m.id_capteur
    JOIN gaz g ON g.id_gaz = c.id_gaz
    JOIN agence a ON a.id_agence = c.id_agence
    JOIN ville v ON v.id_ville = a.id_ville
    JOIN region reg ON reg.id_region = v.id_region
    WHERE g.sigle_gaz = 'HFC'
    GROUP BY reg.id_region, reg.nom_region, DATE_FORMAT(m.date_mesure, '%Y-%m')
) hfc_min
WHERE rang_region = 1
ORDER BY region;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

region	mois	concentration_moyenne_hfc
Auvergne-Rhône-Alpes	2023-06	45.00
Bretagne	2022-08	58.20

La requête permet d’identifier, pour chaque région, le mois où la concentration moyenne de HFC est la plus faible. Elle calcule d’abord la moyenne des valeurs mesurées par région et par mois à l’aide de AVG, puis utilise RANK() pour classer les mois de chaque région du plus faible au plus élevé. La fonction PARTITION BY sépare les calculs par région. Ensuite, la sous-requête sélectionne uniquement les lignes classées en rang 1, ce qui correspond aux concentrations minimales. Enfin, les résultats sont triés par région pour faciliter la lecture.

Requête 9 :

```sql
SELECT g.nom_gaz, AVG(m.valeur_ppm) AS moyenne
FROM mesure m
JOIN capteur c ON m.id_capteur = c.id_capteur
JOIN gaz g ON c.id_gaz = g.id_gaz
JOIN agence ag ON c.id_agence = ag.id_agence
JOIN ville v ON ag.id_ville = v.id_ville
JOIN region r ON v.id_region = r.id_region
WHERE r.nom_region = 'Île-de-France'
AND YEAR(m.date_mesure) = 2020
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nom_gaz	moyenne
Dioxyde de carbone	305.000000

La requête calcule la concentration moyenne de chaque gaz mesuré en Île-de-France pour l'année 2020. Elle utilise plusieurs jointures pour relier les mesures aux capteurs, aux gaz, puis aux régions. Le filtre WHERE permet de ne garder que les données de la région Île-de-France et de l'année 2020. La fonction AVG calcule la moyenne des valeurs en ppm pour chaque gaz, et GROUP BY permet de regrouper les résultats par type de gaz afin d'obtenir une moyenne par gaz.

Requête 10 :

```sql
SELECT a.id_agent, a.nom_agent,
COUNT(r.id_rapport) /
TIMESTAMPDIFF(MONTH, a.date_embauche_agent, CURDATE()) AS productivite
FROM agent a
JOIN agent_administratif ad ON a.id_agent = ad.id_agent
JOIN redige rd ON a.id_agent = rd.id_agent
JOIN rapport r ON rd.id_rapport = r.id_rapport
JOIN agence ag ON a.id_agence = ag.id_agence
WHERE ag.nom_agence = 'Agence Toulouse'
GROUP BY a.id_agent;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_agent	nom_agent	productivite
8	Moreau	0.0270
18	Fournier	0.0179

La requête calcule la productivité des agents administratifs de l'agence de Toulouse. Elle mesure le nombre de rapports rédigés par agent

(COUNT(r.id_rapport)) et le divise par le nombre de mois depuis leur embauche (TIMESTAMPDIFF). Cela permet d'obtenir un indicateur de productivité mensuel. Les jointures relient les agents à leurs rôles administratifs, aux rapports qu'ils ont rédigés et à leur agence. Le filtre WHERE limite les résultats à l'agence de Toulouse, et le GROUP BY permet de calculer la productivité pour chaque agent individuellement.

Requête 11 :

```sql
SET @gaz_recherche = 'CH4';
SELECT DISTINCT
    r.id_rapport,
    r.titre_rapport,
    r.date_publication,
    g.nom_gaz,
    g.sigle_gaz
FROM rapport r
JOIN contient co ON co.id_rapport = r.id_rapport
JOIN mesure m ON m.id_mesure = co.id_mesure
JOIN capteur c ON c.id_capteur = m.id_capteur
JOIN gaz g ON g.id_gaz = c.id_gaz
WHERE g.sigle_gaz = @gaz_recherche
   OR g.nom_gaz = @gaz_recherche
ORDER BY r.date_publication DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_rapport	titre_rapport	date_publication	nom_gaz	sigle_gaz
11	Suivi CH4 mai juin 2023	2023-07-15	Méthane	CH4

La requête permet de récupérer les rapports liés à un gaz précis, ici le CH_4 , en utilisant une variable @gaz_recherche pour rendre la recherche flexible. Elle relie les rapports aux mesures via plusieurs jointures afin d'identifier les gaz associés. Le filtre WHERE permet de chercher le gaz soit par son sigle (sigle_gaz) soit par son nom (nom_gaz). DISTINCT est utilisé pour éviter les doublons si un même rapport contient plusieurs mesures du même gaz. Enfin, les résultats sont triés par date de publication décroissante pour afficher les rapports les plus récents en premier.

Requête 12 :

```sql
SELECT
    reg.nom_region,
    COUNT(DISTINCT c.id_capteur) AS nombre_capteurs,
    COUNT(DISTINCT agt.id_agent) AS nombre_personnel_agence
FROM region reg
JOIN ville v ON v.id_region = reg.id_region
JOIN agence a ON a.id_ville = v.id_ville
LEFT JOIN agent agt ON agt.id_agence = a.id_agence
LEFT JOIN capteur c ON c.id_agence = a.id_agence
GROUP BY reg.id_region, reg.nom_region
HAVING COUNT(DISTINCT c.id_capteur) > COUNT(DISTINCT agt.id_agent)
ORDER BY reg.nom_region;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nom_region	nombre_capteurs	nombre_personnel_agence

La requête compare, pour chaque région, le nombre de capteurs installés et le nombre d'agents travaillant dans les agences de cette région. Elle utilise plusieurs jointures pour relier les régions aux villes, aux agences, puis aux agents et aux capteurs. Les LEFT JOIN permettent de conserver les régions même si certaines n'ont pas de capteurs ou d'agents. Le GROUP BY regroupe les données par région, tandis que COUNT(DISTINCT…) évite de compter plusieurs fois un même agent ou capteur. La clause HAVING sert à filtrer uniquement les régions où le nombre de capteurs est supérieur au nombre d'agents.

Exigence demander :

Il nous a été demander dans les spécifitiations technique d'utiliser un moteur de stockage nous avons opter pour InnoDB il va déterminer la manière dont la base de données enregistre et manipule les données. Ici InnoDB est un moteur fiable qui gère les transactions, les relations entre les tables et assure la sécurité des données.

Cela va éviter de dépendre du moteur de stockage de Workbench ce qui rendra le code plus sûr, clair et fiable

Il a aussi fallu réaliser deux comptes utilisateur, un qui est admin, qui possède donc tous les droits sur la base de données et l'autre qui est uniquement un compte user

Pour la réalisation de cette demande voici le code

```
2      -- 1. CREATION DES COMPTES
3
4
5  ●   CREATE USER 'admin'@'localhost' IDENTIFIED [redacted]
6  ●   CREATE USER 'user'@'localhost' IDENTIFIED [redacted]
7
8      -- 2. DROITS ADMIN (TOTAL)
9
10
11 ●   GRANT ALL PRIVILEGES ON livrable2.* TO 'admin'@'localhost' WITH GRANT OPTION;
12
13
14     -- 3. DROITS USER (LIMITÉS)
15
16
17 ●   GRANT SELECT, INSERT, UPDATE ON livrable2.* TO 'user'@'localhost';
18
19
20     -- 4. APPLICATION DES DROITS
21
22
23 ●   FLUSH PRIVILEGES;
```

Ici nous utilisons la fonction **create** user pour la création des deux comptes

La partie **Grant** va définir les droits accorder à l'utilisateur

- All privilèges: elle va accorder à l'user tous les droits que ce soit modification, suppression etc...
- Grant: avec Select, Insert et Update va accorder à l'utilisateur le droit unique de pouvoir modifier afficher et inséré des informations dans la BDD, il ne pourra pas supprimer ou autre

La fonction de fin **Flush privileges** va forcer MySQL à lire ces privilèges, car il se peut qu'ils ne les prennent pas en compte.

Conclusion :

En somme nous avons à la suite de ce premier livrable apporter les informations nécessaires aux tables crée précédemment, pour assurer la cohérence de réponse lors de l'exécution des requêtes exiger pour ce livrable